

- It push the max to the last by adjacent swap.

1) 13 52 46 20 9 24
↓
no swap

2) 13 52 46 20 9 24
swap

3) 13 46 52 20 9 24
swap

4) 13 46 20 52 9 24
swap

5) 13 46 20 9 52 24
swap

6) 13 46 20 9 24 52
No Swap

7) 13 46 20 9 24 52
swap

8) 13 20 46 9 24 52
swap

9) 13 20 9 46 24 52
swap

10) 13 20 9 24 46 52
No swap

11) 13 20 9 24 46 52
swap

12) 13 9 20 24 46 52
No swap

13) 13 9 20 24 46 52
swap

14) 9 13 20 24 46 52
No swap
No swap

Thus Sorted.

class Solution:

```
def bubbleSort(self, arr):
```

```
    # code here
```

```
    for i in range(len(arr)-1, -1, -1):
```

swapped = 0 → Because of this variable & last two lines if already sorted no's takes $O(N)$

```
        for j in range(0, i):
```

```
            if arr[j] > arr[j+1]:
```

```
                arr[j], arr[j+1] = arr[j+1], arr[j]
```

```
                swapped += 1
```

```
        if swapped == 0: →
```

```
            break
```

Time Complexity

Best Case = $O(N)$ — when array is already sorted.

Worst Case -

Worst Case

0

0

Worst Case -

$$(n-1) + (n-2) + (n-3) + \dots + 2$$

$$= \frac{n(n+1)}{2} = \underline{O(n^2)}$$